

Kairos: A Pyramidal Codec and Diffusion over One Byte Space

Bidirectional Delta/Window-Attention Mixture-of-Experts – Does It Converge? Sanity Checks at Tiny Scale

Fabien Furfaro*
September 2026

Abstract

Background: Small language models trained on a single local machine leave little room for architectural trial and error. Kairos combines bidirectional gated-DeltaNet linear attention and sliding-window attention over shared projections (LiZAttention2), block-wise Attention-Residual aggregation, a sparse Mixture-of-Experts (MoE) feed-forward layer and a multi-scale byte-level codec, trained with a uniform-noise denoising objective over one byte space shared by text, image, audio, video, lidar and control streams; this report asks whether the combination optimizes at all before asking whether it is useful. **Methods:** We give code-derived algorithms for each component, including a cross-session recurrent-state memory gate, then run convergence sanity checks: overfitting a single 440-byte sentence at 1–15M parameters, text only, with three seeds for codec and attention choices and single-run sweeps for seven further factors. **Results:** The two attention variants tested at 3 000 steps (LiZAttention2 and window attention alone) reach a minimum loss near 10^{-3} . After 600 steps, the mean minimum loss ranges from 0.018 (patch codec) to 0.354 (depthwise convolutional codec) with an identical ranking on three seeds. An apparent advantage of shared over separate attention projections closes with more steps or more diverse data and reverses when the projections are frozen: it is a convergence-speed effect, not a capacity effect. **Conclusions:** These checks suggest that the implementation optimizes; they say nothing about generalization, multimodal training or comparison with existing models, which a planned pretraining run is meant to address. The Discussion sizes a 200M-total/25M-active target analytically; none of its throughput figures is measured.

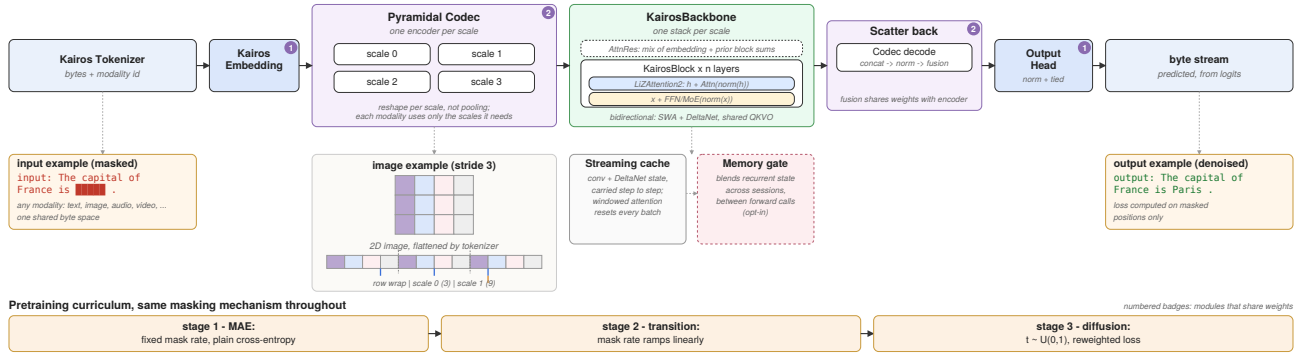


Figure 1: Kairos data and training flow, left to right. The tokenizer (§3.1) flattens every modality to one byte stream; the image example shows a 2D image flattened row by row (its <ENDLINE> markers are omitted). Each KairosBlock is a standard pre-norm attention+FFN residual pair; AttnRes only re-mixes what feeds into it. Training (bottom) uses one masking mechanism across three curriculum stages (MAE → transition → diffusion), illustrated with a masked-input/denoised-output example.

*Code and configurations: <https://github.com/fabienfrfr/Kairos>

1 Introduction

Small, edge-deployable language models (MobileLLM [27], SmolLM2 [3], Gemma 3 [18]) are routinely trained on far more tokens than the Chinchilla-optimal ratio [21], on the premise that inference cost dominates total cost of ownership. That premise says nothing about training cost when the training hardware is itself constrained, the setting of this report. Three architectural levers target efficiency in largely separate literatures: (i) sub-quadratic sequence mixing, through local windowed attention [6, 24] and linear state-compressing mixers derived from state-space models (Mamba [19]) and the delta rule [43, 44]; (ii) sparse capacity scaling through MoE routing [9, 11, 13, 38]; (iii) non-autoregressive discrete diffusion training [4, 28, 31, 37], which generalizes masked language modeling [12]. A fourth, orthogonal choice removes the tokenizer: byte-level inputs [35, 42, 45] give one vocabulary that can cover several modalities, an idea we had explored at smaller scale for two modalities [14] and revise substantially here for six.

Each lever is usually reported on its own. Combining them in one model small enough to train on a single local machine raises a practical question before any question of quality: does the combination optimize at all? Hybrid attention, sparse routing, a multi-scale codec and a corruption-based objective each add moving parts that could prevent optimization, and a memorization test is the cheapest way to check. **Research question:** can a Kairos network drive the denoising loss of a fixed byte string to near zero, and which design choices change how fast?

In this paper, we specify the Kairos architecture from its implementation (Figure 1) and report convergence sanity checks at 1–15M parameters. These checks are necessary and not sufficient: they do not measure generalization. Held-out training and a comparison with Pythia-14M [7] are planned and not reported (§6.2).

Our contributions are:

1. The Kairos architecture (Figure 1), specified from its implementation: the multimodal byte tokenizer (§3.1), PyramidalCodec (Algorithm 1), LiZAttention2 (Algorithm 2), Block-AttnRes (Algorithm 3), sparse MoE (§3.5), a cross-session memory gate (Algorithm 4) and the denoising objective with its curriculum (Algorithm 5), each adapted from a specific prior method or, for the codec and the memory gate, combining existing ideas.
2. A convergence study at tiny scale (§5.1) with 3-seed statistics for codec and attention choices and single-run sweeps for seven further factors.
3. A worked correlation-versus-causation case (§5.2): a clear single-run effect that does not survive three controls, and a recommendation to report steps-to-threshold rather than loss at a fixed step.

The Discussion also sizes an untrained 200M-total/25M-active target with a roofline argument (§6); it is analytical and not measured.

2 Related Work

Efficient attention. Full self-attention [40] is quadratic in sequence length. The delta rule [43, 44] improves plain linear attention with an error-correcting state update; Kairos runs it *bidirectionally* (a forward pass and a time-reversed pass, concatenated) to match the context a denoising objective needs. Sliding-window attention [6, 24] is likewise bidirectional here, with rotary position embeddings [39]; the code supports grouped-query attention [2], which the configurations reported here do not exercise. Coupling a delta-rule branch and a window-attention branch through shared projections follows TPTT [15].

Byte-level, hierarchical and multimodal models. ByT5 removes the subword tokenizer [42]; Kairos’ tokenizer subclasses it and extends it to other modalities. MegaByte models bytes with a global model over

fixed-size patches and a local model inside each patch [45]; the Byte Latent Transformer forms patches dynamically from next-byte entropy [35]; Hourglass Transformers shorten and re-expand the sequence inside one stack [30]; Perceiver IO reads arbitrary input arrays, including raw bytes, through cross-attention into a fixed latent array [23]. Kairos’ PyramidalCodec instead keeps S parallel views of one sequence at geometrically growing patch sizes, runs a backbone on each, and fuses them after upsampling.

Sparse MoE. MoE architectures [9, 11, 13, 38] decouple capacity from per-token compute. **KairosMoE** subclasses DeepSeek-V3’s routed-expert layer [11] (sigmoid routing scores, shared experts, and load balancing by a per-expert selection bias rather than an auxiliary loss), and **KairosFFN**, used when MoE is disabled, subclasses Qwen2’s dense MLP [36]. Only the expert-weight initialization is patched, because the upstream implementation used leaves those weights uninitialized.

Cross-layer residual aggregation. Block-AttnRes replaces the fixed, unit-weight residual accumulation standard since ResNet [20] with the learned, block-windowed softmax attention over earlier layer outputs of Kimi Team [26], used here as published (§3.4).

Discrete diffusion language modeling. Kairos’ corruption is closest to the uniform-replacement transition of D3PM [4]: corrupted positions are filled with tokens drawn uniformly from the vocabulary, and the model is not told which positions were corrupted, unlike the absorbing mask state of MDLM [37]. The loss is weighted by $1/p$ in the manner of MDLM, with a clip, and trained with self-conditioning [8] and a three-stage curriculum (§3.7).

Recurrent memory. Transformer-XL carries hidden states across segments [10], Titans learn a test-time memory [5], and TPTT retrofits such memory into pretrained transformers [15]. Kairos’ gate is simpler: it seeds the recurrent state of each DeltaNet layer for a new sequence from a learned blend of states left by earlier sequences (§3.6).

Our positioning. Kairos assembles components from the works above and adds two elements: a pyramidal codec that runs the backbone on several patch scales, and a cross-session memory gate. We did not perform a systematic literature search and claim no novelty beyond this combination. The aim of this report is to check that the combination trains, not that it outperforms.

3 Model

Figure 1 gives the data flow. The subsections follow it from input bytes to output logits, then the training objective.

3.1 Multimodal byte tokenizer

The tokenizer subclasses the ByT5 tokenizer [42]. Text is UTF-8 and every byte is one id, with the same byte-to-id offset in all modalities, so one vocabulary of 290 entries (3 ByT5 special ids, 256 byte values, 31 added markers) serves every modality. A sequence is a list of typed segments, each wrapped in modality delimiters (for example `<IMG>...</IMG>`) and optionally in channel delimiters (R, G, B, left, right). Encoding returns three aligned streams: the ids, a modality id per position, which drives the scale router of the codec, and an *octet-family* id per position, which names the byte plane a byte belongs to. Families live in a separate stream rather than in the vocabulary, so one byte value can carry different meanings in different planes (18 family ids, family 0 being text and markers).

Modality	Byte serialization	Markers	Families
Text	UTF-8 bytes	none	1
Image	interleaved R, G, B bytes, row by row	<ENDLINE> per row	3
Video	image frames in sequence	<ENDLINE>, <ENDFRAME>	3 (shared)
Audio	2-byte planes of decimated samples	<TICK> every 16,000 samples	2
Lidar	(x, y, z , intensity), 2 bytes each	<PTSEP> every 32 points	8
Control	action and state, 2 bytes each, interleaved	<OBS> for state-only	4

Table 1: Every modality reduces to bytes plus at most three kinds of side information: a delimiter, a structural marker and a family id (`kairos/tokenizer.py`). The control serialization follows the action-state encoding of Furfaro [14].

Non-text values are serialized without a header (Table 1). A float $x \in [\ell, u]$ becomes n big-endian byte planes, as the digits of a place-value number (Eq. 1):

$$q = \left\lfloor \text{clip}\left(\frac{x-\ell}{u-\ell}, 0, 1\right) (256^n - 1) \right\rfloor, \quad b_k = \left\lfloor q/256^{n-1-k} \right\rfloor \bmod 256, \quad (1)$$

with $n = 2$ by default and fixed ranges ($[-1, 1]$ for audio and control signals, ± 100 for lidar coordinates, $[0, 1]$ for intensity), so the encoding needs no per-sample header. Optional downsampling before quantization (block mean for pixels, decimation by 4 for audio and signals by default, point stride for lidar) makes it lossy. Decoders recover shapes by counting markers and raise an error on a malformed stream, so a generated stream can be checked.

3.2 Embedding and PyramidalCodec

Each token id and its octet-family id are embedded and fused by a linear layer (§3.1); the output head, tied to the token embedding, predicts both the byte and its family.

The PyramidalCodec builds S parallel views of the sequence with patch sizes $p_s = \text{stride}^s$ (notebook config: stride 3, $S = 4$, so $p = 3, 9, 27, 81$). A shared rank- r bottleneck mixer ($\text{Conv1d}_{d \rightarrow r} \rightarrow \text{GELU} \rightarrow \text{Conv1d}_{r \rightarrow d}$, $r = \max(8, d/4)$, cost $O(rd)$ against $O(d^2)$ for a dense 1×1 convolution) supplies cross-channel context once; each scale then applies a grouped convolution with kernel and stride p_s and its own mixer (Algorithm 1). The group count is the one knob that trades compute for expressiveness: `channels_per_group=1` (default) is depthwise and cheapest, and raising it toward d approaches a dense convolution. `mode="patch"` replaces the convolution and mixer chain by one $\text{Linear}(p_s d \rightarrow d)$ per scale, decoded with the transposed weight (tied encode and decode). Each scale runs through its own backbone, or through one shared backbone (about 75% fewer backbone parameters for $S = 4$). A scale router lets each modality read only a subset of scales (notebook config: text $\rightarrow$ scales 1–2, control 1, image 2–3, lidar 2, audio 3, video 3–4, metadata 4); positions of modalities not routed to a scale bypass its backbone. Decoding mirrors encoding with grouped transposed convolutions and per-scale mixers, then concatenates the scales, applies RMSNorm and fuses them with a linear layer.

3.3 LiZAttention2: shared-projection bidirectional hybrid attention

Each layer runs a bidirectional sliding-window branch (mask $|q - k| \leq w$, RoPE) and a bidirectional gated-DeltaNet branch over the same normalized input. The DeltaNet branch’s Q/K/V and output projections are deleted and replaced by references to the window branch’s, so both mixers read the same linear projections of x (`liz2_share_qkv=False` restores independent ones; §5.2). The DeltaNet branch keeps its own value expansion (factor 2), depthwise causal convolution, decay and write-strength gates, output gate, and a merge of its forward and time-reversed passes (Algorithm 2). The two branch outputs are concatenated, normalized with RMSNorm [46] and projected back to d .

Algorithm 1 PyramidalCodec, `mode="conv"`: encode, per-scale backbone, decode

Require: $x \in \mathbb{R}^{B \times L \times d}$; patch sizes $p_1, \dots, p_S$; backbones $f_1, \dots, f_S$

- 1: $h \leftarrow \text{Mixer}_{\text{pre}}(x)$ ▷ shared channel mixing, computed once
 - 2: **for** $s = 1$ **to** S **do**
 - 3: $h_s \leftarrow \text{GroupedConv1d}(\text{PadToMultiple}(h, p_s); \text{kernel} = \text{stride} = p_s)$
 - 4: $z_s \leftarrow \text{Mixer}_s(h_s)$ ▷ $\lceil L/p_s \rceil$ positions
 - 5: $\tilde{z}_s \leftarrow f_s(z_s)$ ▷ active positions only, chosen by the scale router
 - 6: $u_s \leftarrow \text{Trim}_L(\text{Mixer}'_s(\text{ConvTranspose1d}(\tilde{z}_s)))$
 - 7: **end for**
 - 8: **return** $\text{Linear}_{Sd \rightarrow d}(\text{RMSNorm}(\text{concat}(u_1, \dots, u_S)))$
-

Algorithm 2 LiZAttention2 forward (one layer)

Require: $x \in \mathbb{R}^{B \times L \times d}$; shared projections W_q, W_k, W_v, W_o

- 1: $o^{\text{swa}} \leftarrow W_o \text{BidirWindowAttn}(\text{RoPE}(W_q x), \text{RoPE}(W_k x), W_v x; w)$
 - 2: **for** $\text{dir} \in \{\text{fwd}, \text{bwd}\}$ **do**
 - 3: $x' \leftarrow x$ if fwd, else $\text{flip}(x)$ ▷ flip along the sequence axis
 - 4: $(q, k, v) \leftarrow \text{SiLU DepthwiseCausalConv}([W_q x'; W_k x'; E_v W_v x'])$ ▷ E_v : value expansion
 - 5: $\beta \leftarrow \sigma(W_\beta x')$; $g \leftarrow -\exp(A_{\log}) \text{softplus}(W_a x' + b_{dt})$ ▷ per head
 - 6: $o^{\text{dir}} \leftarrow \text{GatedDeltaRule}(q, k, v; g, \beta) \odot \text{SiLU}(W_g x')$ ▷ q, k L2-normalized
 - 7: **end for**
 - 8: $o^\Delta \leftarrow W_o W_{\text{lr}} \text{RMSNorm}([o^{\text{fwd}}; \text{flip}(o^{\text{bwd}})])$
 - 9: **return** $W_{\text{mix}} \text{RMSNorm}([o^{\text{swa}}; o^\Delta])$ ▷ $2d \rightarrow d$
-

3.4 Block-AttnRes aggregation

Instead of the plain residual stream $x_{i+1} = x_i + f_i(x_i)$, each layer’s *input* is a learned, softmax-weighted mix of the embedding, the finalized block sums of earlier layers, and a running partial sum of the current block [26] (Algorithm 3); the sublayers of f_i still use ordinary residual connections around their own input. The weight vector w of each aggregator is initialized to zero, so the initial mix is uniform. With block size 1 each earlier layer output is a separate source, a densely connected pattern [22] with learned softmax weights; with block size n_{layers} the sources reduce to the embedding and one running sum, and larger sizes change nothing. The notebook config uses block size 4 over 4 layers; the ablation networks have one layer, where the aggregator is a structural no-op.

3.5 Sparse Mixture-of-Experts

Each token is routed to k of E routed experts plus one or more always-on shared experts. Balance is maintained by nudging a per-expert selection bias toward equal load (DeepSeek-V3 style, no auxiliary loss by default). The active-parameter count used throughout is computed directly:

$$N_{\text{act}} = N_{\text{total}} - N_{\text{expert}} + N_{\text{expert}} \frac{k}{E}, \quad (2)$$

where N_{expert} sums the parameters under modules whose name contains `".experts."`; shared experts, the router, the attention and DeltaNet path and the codec count as fully active (`count_active_parameters`).

3.6 KairosMemoryGate: cross-session recurrent-state memory

With `use_memory_gate=True`, the recurrent state of every DeltaNet layer of every scale is seeded for a new sequence not from zero but from a gate over a bank of states left by earlier sequences (the preceding batch’s final state, or an externally supplied bank). One gate module is shared by all scales and layers.

Algorithm 3 Block-AttnRes backbone forward

Require: embedding x_0 ; layers $f_1, \dots, f_L$; block size S ; learned $w_i \in \mathbb{R}^d$ per aggregator

```
1: completed  $\leftarrow []$ ; partial  $\leftarrow \text{None}$ ; count  $\leftarrow 0$ 
2: for  $i = 1, \dots, L$  do
3:   sources  $\leftarrow [x_0] \cup \text{completed} \cup (\{\text{partial}\} \text{ if not None})$ 
4:    $V \leftarrow \text{stack}(\text{sources})$ ;  $K \leftarrow \text{RMSNorm}(V)$ 
5:    $h \leftarrow \sum_{\text{sources}} \text{softmax}(w_i^\top K) \cdot V$   $\triangleright$  softmax over sources
6:    $x_i \leftarrow f_i(h)$ 
7:   partial  $\leftarrow x_i$  if partial is None else partial +  $x_i$ ; count += 1
8:   if count =  $S$  then
9:     completed.append(partial); partial  $\leftarrow \text{None}$ ; count  $\leftarrow 0$ 
10:  end if
11: end for
12: return  $\text{RMSNorm}(x_L)$ 
```

Algorithm 4 Memory gate, applied per (scale, DeltaNet layer)

Require: zero state $s_0 \in \mathbb{R}^{B \times D}$; bank $M \in \mathbb{R}^{n \times D}$ of flattened prior states

```
1: if  $n = 0$  then
2:   return  $s_0$ 
3: end if
4: if  $n = 1$  then
5:   return  $M_1$  broadcast to  $B$  rows
6: end if
7:  $q \leftarrow \text{Linear}_{D \rightarrow r'}(s_0)$ ;  $K \leftarrow [q; \text{Linear}_{D \rightarrow r'}(M)]$ 
8:  $o \leftarrow \text{MultiheadAttention}_{h=1}(q, K, K)$ 
9: return  $\text{Linear}_{r' \rightarrow D}(o)$ 
```

It is a single-head cross-attention over a rank- r' bottleneck, $r' = \max(8, \text{round}(\sqrt{D}/4))$, where D is the flattened DeltaNet state size ($D = 2048$ and $r' = 11$ for the notebook config). The query is the projection of the (zero) current state; the keys and values are that same projection followed by the projected bank, so the gate can attend to the bank or fall back on the current state (Algorithm 4). It is applied to the forward pass only; the time-reversed pass starts from zero.

3.7 Denoising objective and curriculum

Given a clean byte sequence x_0 , each row draws a rate $p \sim U(\epsilon, p_{\max})$ and a Bernoulli(p) mask over non-prompt, non-padding positions. Masked positions are replaced by tokens drawn uniformly from the vocabulary (and random octet-family ids), giving x_t ; no mask indicator is provided. The model predicts x_0 at the masked positions with cross-entropy, weighted per token by $\min(c, 1 + \alpha(1/p - 1))$, a blend between plain cross-entropy ($\alpha = 0$) and full $1/p$ weighting ($\alpha = 1$) in the manner of MDLM [37], clipped at c to bound the variance from rows with small p . With probability 0.5 per batch, self-conditioning [8] feeds a detached, no-gradient first prediction back in as an additive embedding, matching its use in `generate()`. The default is $\epsilon = 10^{-3}$, $c = 3$; an octet-family cross-entropy term with weight 1 is added when family ids are present.

The schedule $(p_{\max}^{(t)}, \alpha^{(t)})$ is a pure function of the global step (`stage_mask_schedule`), so resuming from a checkpoint depends only on the persisted step count. It is flat at (0.3, 0) in the MAE stage, ramps linearly to (1.0, 1) over the transition stage, then stays flat in the diffusion stage. The staging was introduced after an informal observation of optimization instability when sampling the full masking range from a random initialization; that observation is not quantified here, and no run without the curriculum

Algorithm 5 Denoising training step (`compute_loss`)

Require: batch x_0 , prompt lengths, pad mask; schedule values $(p_{\max}^{(t)}, \alpha^{(t)})$; clip c

- 1: $p_b \leftarrow \epsilon + (p_{\max}^{(t)} - \epsilon) u_b$, $u_b \sim U(0, 1)$ $\triangleright$ one rate per row
 - 2: $m \leftarrow \text{Bernoulli}(p_b)$ restricted to non-prompt, non-pad positions
 - 3: $x_t \leftarrow x_0$ with $x_t[m] \leftarrow$ tokens drawn uniformly from the vocabulary
 - 4: **if** $u < 0.5$, $u \sim U(0, 1)$ **then** $\hat{y} \leftarrow \text{stopgrad}(\text{model}(x_t))$; add $\text{softmax}(\hat{y})E$ to the embeddings
 - 5: **end if**
 - 6: $\text{logits} \leftarrow \text{model}(x_t, \text{logits_mask} = m)$ $\triangleright$ project masked positions only
 - 7: $\ell_i \leftarrow \text{CE}(\text{logits}_i, x_{0,i}) \cdot \min(c, 1 + \alpha^{(t)}(1/p_{b(i)} - 1))$
 - 8: **return** $\text{mean}_i \ell_i$ $\triangleright$ + octet-family CE when family ids are present
-

	Ablation baseline	Notebook config	Target (sizing only)
d_{model} ; heads (head dim)	64; 4 (16)	64; 4 (16)	—
Backbone layers	1	4	—
Codec stride; scales S	5; 4	3; 4	—
Codec mode	conv, 1 channel per group	conv, 8 channels per group	—
AttnRes block size	1	4	—
Feed-forward	dense, $I = 2048$	7 routed ($k = 1$) + 1 shared, $I = 544$	32 routed ($k = 4$)
Backbone shared across scales	no	yes	—
Memory gate	off	on	—
N_{total}	1.98M	$\sim 14\text{--}15\text{M}$	200M
N_{act}	N_{total}	$N_{\text{total}} - 2.51\text{M}$ (Eq. 2)	25M
Training data	16 identical sentences, 600 steps	sanity checks	30B tokens (budget)

Table 2: The three scales this report distinguishes. Dashes: the target design point is specified only by its parameter and token budget. The notebook config’s 2.51M is derived from its configuration ($4 \times 7 \times 3 \times 64 \times 544 = 2.92\text{M}$ routed-expert parameters, of which 6/7 are inactive); the instantiated count has not been logged. The ablation networks are far smaller than the notebook config, so ablation findings are not statements about the notebook config.

is reported.

4 Experimental Protocol

This section fixes the three model scales, then the overfitting protocol behind every measured number in §5.1.

4.1 Three scales, kept apart

The codebase is used at three scales that this report does not conflate (Table 2). The **ablation networks** of §5.1 use the library defaults of `configs/ablations/baseline.yaml` (1 layer, dense feed-forward). The **Notebook configuration** is the default of the project’s notebook (`notebook/kairos_pretraining.py`): a $\sim 14\text{--}15\text{M}$ -parameter network with a 7-expert top-1 MoE, used for pipeline sanity checks. The **target** is a 200M-total/25M-active design point used in §6 purely to size a future run; it has never been instantiated. Because dense parameters (attention, codec, embeddings) are always active, $N_{\text{act}} = N_{\text{total}}/8 + \frac{7}{8}N_{\text{dense}}$ for $E/k = 8$, and 25M is the lower bound reached when all parameters are routed experts.

Configuration	Seed 0	Seed 1	Seed 2	Mean $\pm$ sd	Params
<code>liz2</code> , depthwise codec (baseline)	0.227	0.258	0.576	0.354 ± 0.193	1.98M
<code>vanilla</code> (SWA only), depthwise codec	0.596	0.728	0.798	0.707 ± 0.103	1.81M
<code>conv</code> codec, 8 channels per group	0.106	0.045	0.075	0.075 ± 0.031	2.50M
<code>conv</code> codec, dense (64 channels per group)	0.037	0.024	0.024	0.028 ± 0.007	8.10M
<code>patch</code> codec (one <code>Linear</code> per scale)	0.019	0.016	0.018	0.018 ± 0.002	4.93M

Table 3: Minimum training loss after 600 steps, three seeds (`docs/experiments/ablations_codec_attention.md`, §1). Depthwise saves parameters relative to `patch` (1.98M against 4.93M) at the cost of slower memorization; 8 channels per group recovers most of the gap for about half of `patch`’s parameters (2.50M against 4.93M). The `liz2`-versus-`vanilla` gap is a convergence-speed effect (§5.2).

4.2 Overfitting protocol

The purpose is a convergence check. A network that cannot drive the loss of a fixed byte string to near zero has an optimization or implementation defect; success is necessary and not sufficient for a useful model, since it says nothing about generalization.

All ablations use `kairos overfit` on the configurations of `configs/ablations/` (`kairos==0.4.1.dev46`, `torch==2.14.0`), on CPU with eager attention and the PyTorch fallback DeltaNet path. The data are 16 identical copies of one 440-byte string (the sentence “the quick brown fox jumps over the lazy dog” repeated ten times), truncated to 128 tokens, i.e. about 2.9 repetitions per sequence; the 63-sentence control uses 63 distinct short sentences. Training uses batch size 4, 600 steps unless noted, $p_{\max} = 0.3$ and no reweighting; the attention window is the library default $w = 128$, which exceeds the 26 positions of the finest scale, so the window branch attends globally. Every step draws a fresh corruption, and *min. loss* is the minimum over the run of the per-step training loss, an extreme value of a noisy per-batch quantity. Training and evaluation data coincide, so the study measures memorization speed, not generalization. Runs are not bit-reproducible even at a fixed seed: the baseline at seed 0 gave 0.227 in one run and 0.31 in another. Seeds are 0, 1, 2 where stated; standard deviations are sample standard deviations ($n - 1$).

5 Results

All results are convergence sanity checks (§4.2) on the ablation networks of Table 2: the training data are the evaluation data, so they measure memorization speed, not generalization. §5.1 compares codecs and attention variants, §5.2 reports a single-run effect that did not survive controls, and the last subsection covers the curriculum and the memory gate.

5.1 Convergence of the tiny-scale networks

Table 3 reports the codec and attention choices under three seeds, on the setup of §4.2. Codec ranking is identical on every seed: `patch` < dense `conv` < `conv` with 8 channels per group < depthwise `conv` (`liz2` baseline), from 0.018 to 0.354. The hybrid attention beats SWA alone on every seed (0.227, 0.258, 0.576 against 0.596, 0.728, 0.798), although the mean $\pm$ sd ranges overlap. The notebook config uses `conv` with 8 channels per group, a compromise between the depthwise default and the dense variants.

The seed spread (± 0.03 to ± 0.19) sets the noise floor for the single-run sweeps of Table 4, so differences of a few hundredths there are not readable. Monotone trends are consistent with the expected direction but confounded with the setup: at 128 tokens a sequence holds about 2.9 verbatim repetitions of the sentence (1.5 at 64, 5.8 at 256), so sequence length and the number of codec scales change how many copies are available for in-context recovery of masked bytes, and the loss reflects copying as much as memorization.

Factor (base configuration)	Min. loss by setting (single run)
Depth n_{layers} (<code>liz2</code> baseline)	1: 0.227; 2: 0.843; 4: 0.368; 8 [†] : 1.228
MoE routing, 7 experts	top-1: 0.281; top-2: 0.272
Reweighting clip c (diffusion regime, <code>conv</code> , 8 channels per group)	$c = 3$: 0.139; $c = 30$: 0.249
AttnRes block size (4 layers)	4: 0.386; 8: 0.386; 16: 0.386
Codec scales S	2: 0.430; 3: 0.308; 4: 0.227
Sequence length (tokens)	64: 1.129; 128: 0.227; 256: 0.035
Batch size	1: 0.890; 4: 0.227; 16 (whole set): 0.032

Table 4: Single-run sweeps; read as indicative (§1 of the log gives the noise floor). [†]300 steps because of compute budget. The default $c = 10$ is omitted: the log has no per-run record for it. Depth does not help memorization and top-1 and top-2 routing are indistinguishable at this noise level.

Check	Shared	Separate	Outcome
600 steps, 16 identical sentences	0.369	0.610	shared ahead
3 000 steps, same data	0.0011	0.0011	gap closes
63 distinct sentences, 600 steps	2.269	2.285	gap closes
QKV frozen after step 100	0.705	0.521	gap reverses

Table 5: Min. loss for shared and separate projections (`docs/experiments/ablations_codec_attention.md`, §7; per-step curves in `docs/experiments/liz2_shared_vs_separate_followup_raw.jsonl`). The advantage was faster early convergence, not higher capacity: with enough steps or diverse data both variants reach the same floor.

A batch of 16 covers the whole training set. The attention-block-size result is a saturation effect: sizes at or above the number of layers give the same computation.

5.2 A correlation-versus-causation case: shared versus separate QKV

Sharing the Q/K/V/O projections between the two attention branches (§3.3) first looked like a clear architectural win: with sharing disabled, min. loss rose from 0.227–0.369 to 0.610–0.858 across repeated 600-step runs. Three controls were run before accepting this as an architectural effect (Table 5).

The same check applies to `liz2` against `vanilla`: at 3 000 steps `vanilla` reaches 0.0014, matching `liz2`’s 0.0011. Computing steps-to-threshold from the saved curves shows that the shared-versus-separate gap is concentrated in the 0.5–0.1 loss range (+9 to +14% steps for separate projections) and vanishes elsewhere, which a single comparison at a fixed step count would have hidden. A plausible mechanism is that shared projections receive gradient from both branches at every step, an effective higher early learning rate. The freezing control is consistent with it but does not isolate it; a direct test would rescale the learning rate of the separate variant. The lesson generalizes: a single-run result on a tiny memorization task is a hypothesis until it survives a step-budget control, a data-scale control and, where a mechanism is suspected, a control that isolates it.

5.3 Curriculum and memory gate

Table 6 splits one 600-step run (baseline, seed 0) by curriculum stage (200 steps each). Losses are not comparable across stages because the objective changes with (p_{max}, α) ; the higher tail mean of the transition stage is consistent with the reweighting ramp raising the loss scale. This is a single seed, and no run without the curriculum exists.

The memory gate was tested through the full training loop for 3 epochs (12 optimizer steps, so absolute

Stage	First-step loss	Tail-mean loss
MAE ($p_{\max} = 0.3, \alpha = 0$)	8.663	1.485
Transition (linear ramp)	1.419	2.007
Diffusion ($p_{\max} = 1.0, \alpha = 1$)	1.873	1.555

Table 6: Per-stage loss of a single run (`docs/experiments/planned_experiments.md`, item 13); the tail definition is given there.

losses are not comparable to the rest of the study): min. loss 7.512 without the gate, 7.453 with the previous batch’s state only, 7.423 with a random bank, and final loss 12.927, 12.893 and 9.752. Twelve steps cannot separate the variants; we draw no conclusion.

6 Discussion

Summary of findings. The tiny networks reach a near-zero loss on the memorization task, so the implementation optimizes; this does not show that the model learns transferable structure. Taken together, the observations suggest that:

- the codec ranking of Table 3 is stable across three seeds, but it was measured after 600 steps on one repeated sentence and is a memorization-speed ranking;
- the hybrid attention (`liz2`) reaches a lower loss than window attention alone at 600 steps on every seed, and both reach the same floor at 3 000 steps;
- the shared-projection advantage is a convergence-speed effect (§5.2); the mechanism is consistent with the freezing control but not isolated;
- depth, top-1 versus top-2 routing and the AttnRes block size show no measurable effect in this setup, the block-size result being a saturation effect;
- the curriculum and the memory gate have no supporting result: one seed for the former, twelve optimizer steps for the latter.

Practical implications. The ablation runs used a CPU. A memorization test of this kind can be repeated whenever a component changes, but the seed spread (± 0.03 to ± 0.19 in Table 3) limits which differences it can detect.

Compute sizing of the target. The 200M-total/25M-active design point has not been instantiated; a roofline argument [41] indicates what a run would need. Training costs $6N_{\text{act}}D = 4.5 \times 10^{18}$ FLOPs for $D = 30\text{B}$ tokens [25], a ratio $D/N_{\text{act}} = 1,200$ far below that of the dense Pythia-14M, trained on about 300B tokens [7]. Chinchilla is fitted on dense models of 70M parameters and more [21], so applying it to 25M active parameters is an extrapolation, and MoE scaling laws [1, 29] parameterize loss rather than step time. Under balanced routing, a microbatch of T tokens touches nearly every expert once T exceeds a few times E/k , so a step reads N_{total} weights, not N_{act} ; with bf16 weights the arithmetic intensity is $\mathcal{I}(T) = (N_{\text{act}}/N_{\text{total}})T$ FLOP/byte, and the smallest microbatch that reaches the ridge point $\mathcal{I}^* = \pi/b_s$ of a GPU with peak throughput π and bandwidth b_s is

$$T_{\min} = \frac{N_{\text{total}}}{N_{\text{act}}} \mathcal{I}^*. \quad (3)$$

With the vendor figures of a T4, an A100 and an H100 [32–34] and $N_{\text{total}}/N_{\text{act}} = 8$, $T_{\min}$ is about 1.6k tokens (T4, A100) to 2.4k tokens (H100), against 200–300 for a dense model of the same size;

below it, throughput is set by N_{total} , not by the active parameters that FLOP-centric accounting counts. Sparsity is therefore not free on memory-limited hardware. These are upper bounds from a model that counts weight traffic only and omits activations, DeltaNet states, codec convolutions, MoE dispatch and communication, and any utilization would be assumed, not measured. Whether a 16 GB card holds a 2.4k-token microbatch of this architecture is an empirical question this report does not answer. The numbers come from `scripts/roofline.py`.

6.1 Limitations

The ablation study memorizes one sentence in 16 copies on text only, with a window branch that attends globally, on CPU, with runs that are not bit-reproducible; its metric is a minimum over noisy per-step losses. Only Table 3 and §5.2 carry multi-seed or multi-control statistics; the other sweeps are single-run. Parameter counts of the ablation rows in Table 3 other than the baseline are carried over from earlier logs. The notebook config’s active-parameter count has not been logged from an instantiated model. No dense baseline is compared, so nothing here supports a claim of superiority; the multimodal, curriculum and memory-gate components have no supporting result yet. The target sizing is analytical.

6.2 Future work

The long-term aim behind Kairos is a small multimodal foundation model that a single local machine can pretrain end to end; this report only asks whether the substrate optimizes. The next step toward that aim is a pretraining run, which would turn these sanity checks into held-out results. The pretraining notebook (`notebook/kairos_pretraining.py`) trains on the text corpus `keep-it-simple` [16] (640,646 rows of bidirectional prompt-text pairs) and on `keep-it-simple-multimodal` [17] (6,336 rows, a deliberately small pilot mix), through the MAE, transition and diffusion curriculum. Planned measurements: (i) held-out loss per curriculum stage and per modality against trivial baselines, and text loss with and without the multimodal data; (ii) a comparison with Pythia-14M [7] in bits per byte, since tokenizers and objectives differ. Equal total parameter counts do not mean equal compute: Pythia’s parameters are largely embeddings of a vocabulary of about 50k entries, whereas a byte-level vocabulary puts nearly all Kairos parameters in the backbone, so total, non-embedding and active parameters and FLOPs per byte should all be reported. A Pythia-shaped model trained on the same data would separate architecture from data, and a held-out corpus that neither model has seen would avoid favoring Kairos; (iii) 3-seed, steps-to-threshold reruns of the single-run sweeps on diverse data; (iv) a run without the curriculum; (v) a measured tokens/s against microbatch size for the 200M/25M and a 50M/25M variant, which tests Eq. 3 directly.

7 Conclusion

Kairos poses a design question at 1–15M parameters: whether a hybrid bidirectional DeltaNet/window-attention, MoE, denoising architecture over a byte-level space optimizes at all. Memorization tests answer yes at this scale. The clearest architectural effect in these tests, a shared-projection advantage, was a convergence-speed artifact, which is why we recommend steps-to-threshold and control runs before reading a tiny-scale comparison. Generalization, multimodal training and comparison with existing models are the objectives of the run described in §6.2; the 200M/25M design point of §6 is a sizing exercise only.

Code and reproducibility

Code, configurations and logs are in the repository. Ablation configurations are in `configs/ablations/`; a run is `kairos overfit configs/ablations/<name>.yaml -steps 600 -mask-p-max 0.3 -no-mask-reweight -seed <0|1|2>`. The sizing numbers of §6 come from `scripts/roofline.py`,

checked by `tests/test_roofline.py` and `tests/test_paper.py`. The pretraining notebook (marimo) is `notebook/kairos_pretraining.py`.

References

- [1] Samira Abnar, Harshay Shah, Dan Busbridge, Alaaeldin Mohamed Elnouby Ali, Josh Susskind, and Vimal Thilak. Parameters vs flops: Scaling laws for optimal sparsity for mixture-of-experts language models. *arXiv preprint arXiv:2501.12370*, 2025.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [3] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Leandro von Werra, Thomas Wolf, et al. Smollm2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- [4] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. *arXiv preprint arXiv:2107.03006*, 2021.
- [5] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [6] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [7] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. *arXiv preprint arXiv:2304.01373*, 2023.
- [8] Ting Chen, Ruixiang Zhang, and Geoffrey Hinton. Analog bits: Generating discrete data using diffusion models with self-conditioning. *arXiv preprint arXiv:2208.04202*, 2022.
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- [10] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.
- [11] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [13] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- [14] Fabien Furfaro. PixelBytes: Catching unified representation for multimodal generation. <https://arxiv.org/abs/2410.01820>, 2024. arXiv:2410.01820.

- [15] Fabien Furfaro. Tptt: Transforming pretrained transformer into titans. *arXiv preprint arXiv:2506.17671*, 2025.
- [16] Fabien Furfaro. keep-it-simple. <https://huggingface.co/datasets/ffurfaro/keep-it-simple>, 2026. Hugging Face dataset.
- [17] Fabien Furfaro. keep-it-simple-multimodal. <https://huggingface.co/datasets/ffurfaro/keep-it-simple-multimodal>, 2026. Hugging Face dataset.
- [18] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- [19] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.
- [21] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [22] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. *arXiv preprint arXiv:1608.06993*, 2016.
- [23] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. Perceiver IO: A general architecture for structured inputs & outputs. *arXiv preprint arXiv:2107.14795*, 2021.
- [24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [25] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [26] Kimi Team. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.
- [27] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. *arXiv preprint arXiv:2402.14905*, 2024.
- [28] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- [29] Jan Ludziejewski, Maciej Pi  ro, Jakub Krajewski, Maciej Stefaniak, Micha   Krutul, Jan Ma  snicki, Marek Cygan, Piotr Sankowski, Kamil Adamczewski, Piotr Mi  s  , and Sebastian Jaszczur. Joint moe scaling laws: Mixture of experts can be memory efficient. *arXiv preprint arXiv:2502.05172*, 2025.
- [30] Piotr Nawrot, Szymon Tworkowski, Micha   Tyrolski,   ukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language models. *arXiv preprint arXiv:2110.13711*, 2021.

- [31] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- [32] NVIDIA. NVIDIA T4 tensor core GPU. <https://www.nvidia.com/en-us/data-center/tesla-t4/>, 2018. Accessed September 2026.
- [33] NVIDIA. NVIDIA A100 tensor core GPU. <https://www.nvidia.com/en-us/data-center/a100/>, 2020. Accessed September 2026.
- [34] NVIDIA. NVIDIA H100 tensor core GPU. <https://www.nvidia.com/en-us/data-center/h100/>, 2022. Accessed September 2026.
- [35] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.
- [36] Qwen Team, An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [37] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.
- [38] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [39] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.
- [41] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- [42] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.
- [43] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- [44] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.
- [45] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MegaByte: Predicting million-byte sequences with multiscale transformers. *arXiv preprint arXiv:2305.07185*, 2023.
- [46] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019.